Noah Xu HW5

```sql
CREATE DATABASE IF NOT EXISTS books_hw5;
USE books_hw5;

TRUNCATE TABLE books;
CREATE TABLE IF NOT EXISTS books (
    bookid INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    category VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

SELECT * FROM books;
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| ✖ | 1 | 23:27:51 | CREATE TABLE IF NOT EXISTS books ( bookid INT PRIMARY KEY, | title VARCHAR(255) NOT NULL, | author VARCHAR(255) NOT NULL, | cate... | Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near... | 0.000 sec |
| ⚠ | 2 | 23:28:09 | CREATE TABLE IF NOT EXISTS books ( bookid INT PRIMARY KEY, | title VARCHAR(255) NOT NULL, | author VARCHAR(255) NOT NULL, | cate... | 0 row(s) affected, 1 warning(s): 1050 Table 'books' already exists | 0.047 sec |
| ✔ | 3 | 23:29:08 | CREATE DATABASE IF NOT EXISTS books_hw5 | | | | 1 row(s) affected | 0.062 sec |
| ✔ | 4 | 23:29:08 | USE books_hw5 | | | | 0 row(s) affected | 0.000 sec |
| ✔ | 5 | 23:29:11 | CREATE TABLE IF NOT EXISTS books ( bookid INT PRIMARY KEY, | title VARCHAR(255) NOT NULL, | author VARCHAR(255) NOT NULL, | cate... | 0 row(s) affected | 0.062 sec |
| ✔ | 6 | 23:29:29 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 7 | 23:29:29 | SHOW DATABASES | | | | OK | 0.000 sec |
| ✔ | 8 | 23:29:31 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 9 | 23:29:31 | SHOW COLUMNS FROM `books_hw5`.`books` | | | | OK | 0.000 sec |
| ✔ | 10 | 23:29:41 | PREPARE stmt FROM 'INSERT INTO `books_hw5`.`books` (`bookid`,`title`,`author`,`category`,`price`) VALUES(?,?,?,?,?)' | | | | OK | 0.000 sec |
| ✔ | 11 | 23:29:41 | DEALLOCATE PREPARE stmt | | | | OK | 0.000 sec |
| ✔ | 12 | 23:29:59 | SELECT * FROM books LIMIT 0, 1000 | | | | 49 row(s) returned | 0.000 sec / 0.000 sec |
| ✔ | 13 | 23:30:27 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 14 | 23:30:27 | SHOW DATABASES | | | | OK | 0.000 sec |
| ✔ | 15 | 23:30:30 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 16 | 23:30:30 | SHOW COLUMNS FROM `books_hw5`.`books` | | | | OK | 0.000 sec |
| ✔ | 17 | 23:30:40 | PREPARE stmt FROM 'INSERT INTO `books_hw5`.`books` (`bookid`,`title`,`author`,`category`,`price`) VALUES(?,?,?,?,?)' | | | | OK | 0.000 sec |
| ✔ | 18 | 23:30:40 | DEALLOCATE PREPARE stmt | | | | OK | 0.000 sec |
| ✔ | 19 | 23:31:29 | TRUNCATE books | | | | 0 row(s) affected | 0.047 sec |
| ✔ | 20 | 23:31:38 | TRUNCATE TABLE books | | | | 0 row(s) affected | 0.157 sec |
| ✔ | 21 | 23:31:43 | SELECT * FROM books LIMIT 0, 1000 | | | | 0 row(s) returned | 0.000 sec / 0.000 sec |
| ⚠ | 22 | 23:31:48 | CREATE TABLE IF NOT EXISTS books ( bookid INT PRIMARY KEY, | title VARCHAR(255) NOT NULL, | author VARCHAR(255) NOT NULL, | cate... | 0 row(s) affected, 1 warning(s): 1050 Table 'books' already exists | 0.015 sec |
| ✔ | 23 | 23:31:54 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 24 | 23:31:54 | SHOW DATABASES | | | | OK | 0.000 sec |
| ✔ | 25 | 23:31:55 | SHOW SESSION VARIABLES LIKE 'lower_case_table_names' | | | | OK | 0.000 sec |
| ✔ | 26 | 23:31:55 | SHOW COLUMNS FROM `books_hw5`.`books` | | | | OK | 0.000 sec |
| ✔ | 27 | 23:31:59 | PREPARE stmt FROM 'INSERT INTO `books_hw5`.`books` (`bookid`,`title`,`author`,`category`,`price`) VALUES(?,?,?,?,?)' | | | | OK | 0.000 sec |
| ✔ | 28 | 23:31:59 | DEALLOCATE PREPARE stmt | | | | OK | 0.000 sec |

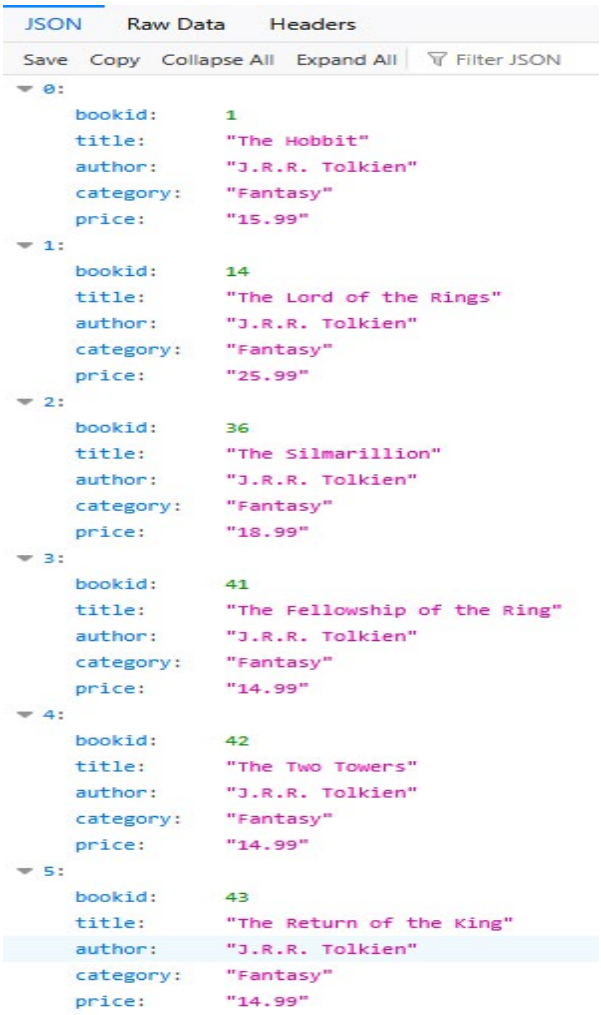
| bookid | title | author | category | price |
|---|---|---|---|---|
| 1 | The Hobbit | J.R.R. Tolkien | Fantasy | 15.99 |
| 2 | 1984 | George Orwell | Dystopian | 12.50 |
| 3 | To Kill a Mockingbird | Harper Lee | Fiction | 14.25 |
| 4 | Dune | Frank Herbert | Sci-Fi | 18.00 |
| 5 | Foundation | Isaac Asimov | Sci-Fi | 16.50 |
| 6 | Pride and Prejudice | Jane Austen | Romance | 10.99 |
| 7 | The Great Gatsby | F. Scott Fitzgerald | Classic | 11.50 |
| 8 | Moby-Dick | Herman Melville | Adventure | 13.99 |
| 9 | War and Peace | Leo Tolstoy | Historical | 19.99 |
| 10 | The Odyssey | Homer | Classic | 12.99 |
| 11 | Crime and Punishm... | Fyodor Dostoevsky | Classic | 14.50 |
| 12 | The Catcher in the ... | J.D. Salinger | Fiction | 10.50 |
| 13 | Brave New World | Aldous Huxley | Dystopian | 13.50 |
| 14 | The Lord of the Rings | J.R.R. Tolkien | Fantasy | 25.99 |
| 15 | Fahrenheit 451 | Ray Bradbury | Dystopian | 11.99 |
| 16 | Jane Eyre | Charlotte Bronte | Romance | 12.00 |
| 17 | The Chronicles of ... | C.S. Lewis | Fantasy | 21.50 |
| 18 | Animal Farm | George Orwell | Dystopian | 9.99 |
| 19 | The Alchemist | Paulo Coelho | Adventure | 14.00 |
| 20 | The Little Prince | Antoine de Saint-... | Fable | 9.50 |
| 21 | Dracula | Bram Stoker | Horror | 11.25 |
| 22 | Frankenstein | Mary Shelley | Horror | 10.75 |
| 23 | The Picture of Dori... | Oscar Wilde | Classic | 11.00 |
| 24 | Les MisÃ©rables | Victor Hugo | Historical | 18.50 |
| 25 | The Count of Mont... | Alexandre Dumas | Adventure | 16.00 |
| 26 | The Brothers Kara... | Fyodor Dostoevsky | Classic | 17.50 |
| 27 | Anna Karenina | Leo Tolstoy | Historical | 15.50 |
| 28 | Catch-22 | Joseph Heller | Satire | 13.25 |
| 29 | The Handmaid's Tale | Margaret Atwood | Dystopian | 14.50 |
| 30 | Neuromancer | William Gibson | Sci-Fi | 15.00 |
| 31 | Snow Crash | Neal Stephenson | Sci-Fi | 16.25 |
| 32 | Hyperion | Dan Simmons | Sci-Fi | 17.00 |
| 33 | Ender's Game | Orson Scott Card | Sci-Fi | 14.75 |

| | | | | |
|---|---|---|---|---|
| 31 | Snow Crash | Neal Stephenson | Sci-Fi | 16.25 |
| 32 | Hyperion | Dan Simmons | Sci-Fi | 17.00 |
| 33 | Ender's Game | Orson Scott Card | Sci-Fi | 14.75 |
| 34 | I Robot | Isaac Asimov | Sci-Fi | 13.00 |
| 35 | The Martian | Andy Weir | Sci-Fi | 15.50 |
| 36 | The Silmarillion | J.R.R. Tolkien | Fantasy | 18.99 |
| 37 | American Gods | Neil Gaiman | Fantasy | 16.50 |
| 38 | Neverwhere | Neil Gaiman | Fantasy | 13.50 |
| 39 | Good Omens | Terry Pratchett | Fantasy | 14.25 |
| 40 | The Name of the ... | Patrick Rothfuss | Fantasy | 17.25 |
| 41 | The Fellowship of t... | J.R.R. Tolkien | Fantasy | 14.99 |
| 42 | The Two Towers | J.R.R. Tolkien | Fantasy | 14.99 |
| 43 | The Return of the ... | J.R.R. Tolkien | Fantasy | 14.99 |
| 44 | Lord of the Flies | William Golding | Classic | 10.25 |
| 45 | The Sun Also Rises | Ernest Hemingway | Classic | 12.25 |
| 46 | A Farewell to Arms | Ernest Hemingway | Classic | 12.75 |
| 47 | The Old Man and t... | Ernest Hemingway | Classic | 9.75 |
| 48 | The Grapes of Wrath | John Steinbeck | Historical | 14.25 |
| 49 | Of Mice and Men | John Steinbeck | Classic | 10.50 |
| 50 | East of Eden | John Steinbeck | Historical | 16.75 |
| NULL | NULL | NULL | NULL | NULL |

http://localhost:3000/?author=J.R.R.%20Tolkien&format=text

| TITLE | AUTHOR | CATEGORY | PRICE |
|---|---|---|---|
| The Hobbit | J.R.R. Tolkien | Fantasy | 15.99 |
| The Lord of the Rings | J.R.R. Tolkien | Fantasy | 25.99 |
| The Silmarillion | J.R.R. Tolkien | Fantasy | 18.99 |
| The Fellowship of the Ring | J.R.R. Tolkien | Fantasy | 14.99 |
| The Two Towers | J.R.R. Tolkien | Fantasy | 14.99 |
| The Return of the King | J.R.R. Tolkien | Fantasy | 14.99 |

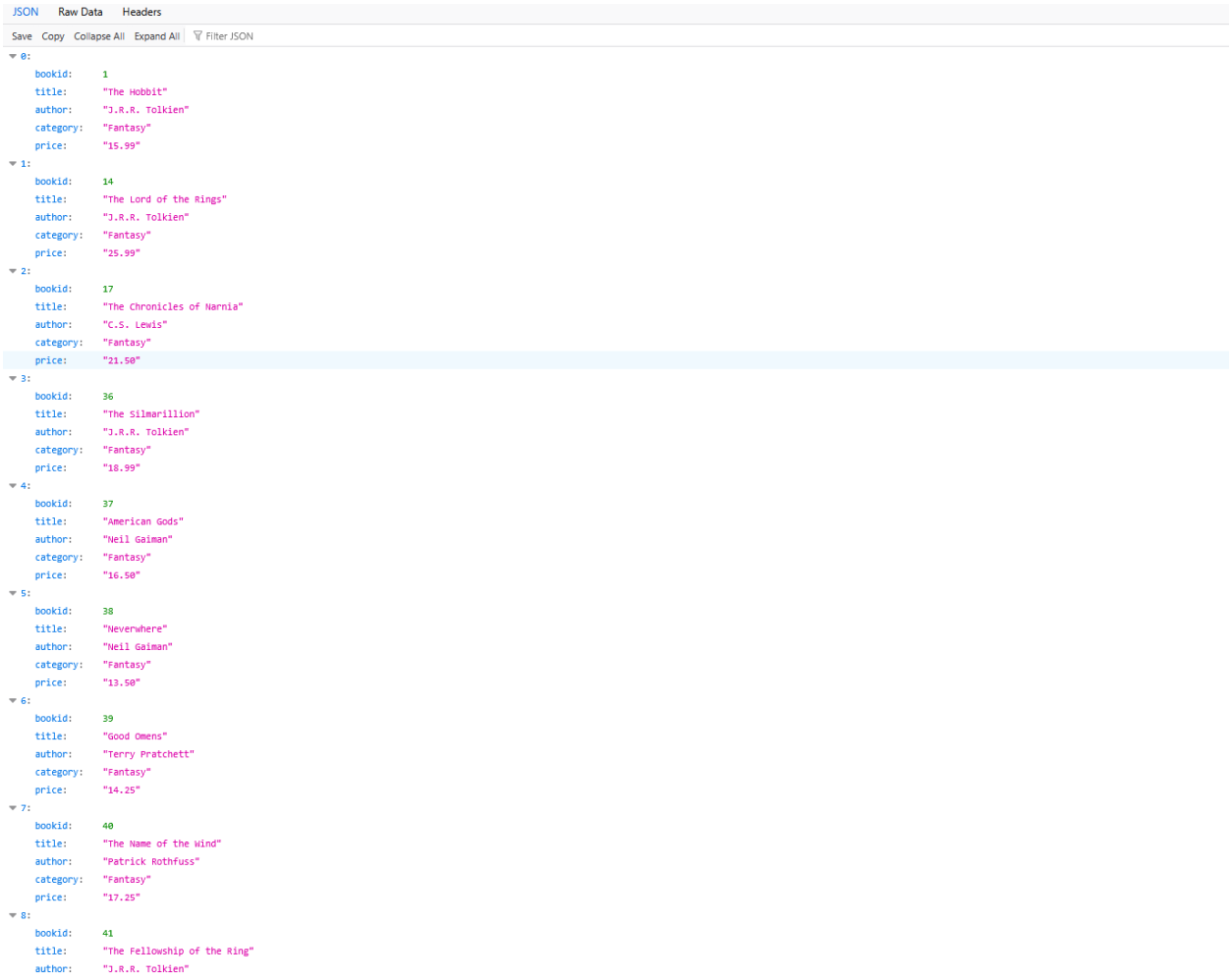
http://localhost:3000/?author=J.R.R.%20Tolkien



http://localhost:3000/?category=Fantasy&format=text

| TITLE | AUTHOR | CATEGORY | PRICE |
|---|---|---|---|
| The Hobbit | J.R.R. Tolkien | Fantasy | 15.99 |
| The Lord of the Rings | J.R.R. Tolkien | Fantasy | 25.99 |
| The Chronicles of Narnia | C.S. Lewis | Fantasy | 21.50 |
| The Silmarillion | J.R.R. Tolkien | Fantasy | 18.99 |
| American Gods | Neil Gaiman | Fantasy | 16.50 |
| Neverwhere | Neil Gaiman | Fantasy | 13.50 |
| Good Omens | Terry Pratchett | Fantasy | 14.25 |
| The Name of the Wind | Patrick Rothfuss | Fantasy | 17.25 |
| The Fellowship of the Ring | J.R.R. Tolkien | Fantasy | 14.99 |
| The Two Towers | J.R.R. Tolkien | Fantasy | 14.99 |
| The Return of the King | J.R.R. Tolkien | Fantasy | 14.99 |

http://localhost:3000/?category=Fantasy


JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
0:
bookid: 1
title: "The Hobbit"
author: "J.R.R. Tolkien"
category: "Fantasy"
price: "15.99"
1:
bookid: 14
title: "The Lord of the Rings"
author: "J.R.R. Tolkien"
category: "Fantasy"
price: "25.99"
2:
bookid: 17
title: "The Chronicles of Narnia"
author: "C.S. Lewis"
category: "Fantasy"
price: "21.50"
3:
bookid: 36
title: "The Silmarillion"
author: "J.R.R. Tolkien"
category: "Fantasy"
price: "18.99"
4:
bookid: 37
title: "American Gods"
author: "Neil Gaiman"
category: "Fantasy"
price: "16.50"
5:
bookid: 38
title: "Neverwhere"
author: "Neil Gaiman"
category: "Fantasy"
price: "13.50"
6:
bookid: 39
title: "Good Omens"
author: "Terry Pratchett"
category: "Fantasy"
price: "14.25"
7:
bookid: 40
title: "The Name of the Wind"
author: "Patrick Rothfuss"
category: "Fantasy"
price: "17.25"
8:
bookid: 41
title: "The Fellowship of the Ring"
author: "J.R.R. Tolkien"

# mysql.js

```
// mysql.js - The Database Connection Pool Module

const mysql = require('mysql2/promise');

// Configuration details
const dbConfig = {
  host: "localhost",
  user: "root",
  password: "hihello123", // !! CHANGE THIS TO YOUR ACTUAL PASSWORD !!
  database: "books_hw5",
  // Connection Pool settings are optional but good practice
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};

// Create and export the connection pool
const pool = mysql.createPool(dbConfig);

// A simple check to ensure the pool is working on startup
pool.getConnection()
  .then(connection => {
    console.log("Database Pool: MySQL connection successful and pool established.");
    connection.release(); // Release the test connection immediately
  })
```

```
  .catch(err => {
    console.error("Database Pool Error: Could not connect to MySQL! Check credentials and if the server is running.", err.message);
  });

module.exports = pool;
```

## search.js

```
//*************************************************************************
//*** set up an HTTP server off port 3000
//*************************************************************************
const express = require('express');
const app = express();
const PORT = 3000;

// Requires the exported pool object from the local 'mysql.js' file
const pool = require('./mysql');

//*************************************************************************
//*** This routing table handles all the GET requests from the browser
//*************************************************************************

// All GET handlers now use async/await by calling the async readAndServe utility

//*************************************************************************
```

```
//*** this handles a specific GET requests from the browser (/)
//*****************************************************************************
app.get("/", async (req, res) => {
  // extract the string received from the browser
  const category = req.query.category;
  const author = req.query.author;
  // return format of html table or json object
  const format = req.query.format;

  // SECURITY FIX: Use prepared statements (the '?' placeholder)
  let sql_query;
  let input;

  if (category) {
    sql_query = "SELECT * FROM books WHERE category = ?";
    input = category;
  } else if (author) {
    sql_query = "SELECT * FROM books WHERE author = ?";
    input = author;
  } else {
    return res.status(404).send("Please provide category or author");
  }

  try {
    const [rows] = await pool.execute(sql_query, [input]);
```

```
    if (format == "text") {
      let html_body = "<HTML><STYLE>body{font-family:arial}</STYLE><BODY><TABLE BORDER=1>";
      html_body += "<TR><TH>TITLE</TH><TH>AUTHOR</TH><TH>CATEGORY</TH><TH>PRICE</TH></TR>";
      for (let i = 0; i < rows.length; i++) {
        html_body += `<TR><TD>${rows[i].title}</TD><TD>${rows[i].author}</TD><TD>${rows[i].category}</TD><TD>${rows[i].price}</TD></TR>`;
      }
      html_body += "</TABLE></BODY></HTML>";
      res.send(html_body);
    } else {
      res.json(rows);
    }
  } catch (err) {
    console.error("SQL Error in / GET:", err);
    res.status(500).send("<html><body><h1>Database Query Error</h1><p>An error occurred while fetching data: " + err.message + "</p></body></html>");
  }
});

// --- Global Error Handler Middleware ---
// Kept as a fallback for non-route-specific errors or errors passed by next()
app.use((err, req, res, next) => {
  // Log the error stack for debugging purposes
  console.error(err.stack);
```

```
  res.status(500).json({ error: 'An unexpected server error occurred.', details:
err.message });
});


//*** set up an HTTP server off port 3000
app.listen(PORT, function () {
  console.log("NodeJS app listening on port " + PORT);
});
```

**wasn't sure if • The complete **Node.js** code implementation for your server. meant pasting code in here when it would be pushed with github repo anyway, but figured safer to include both.